이미지 필터링과 엣지 검출

OpenCV · 블러 · Sobel · Canny

cchyun@gmail.com

Part 1. 이미지 필터링 기초

필터링(컨볼루션)이란 무엇인가?

- **필터링은 이미지의 특정 특징을 강조하거나 원치 않는 부분을 제거하는 과정**

  - **직관적 이해: '주변을 살펴보고 현재 내 값을 결정하기'**
    - 각 픽셀을 하나씩 지나가면서 해당 픽셀과 주변 픽셀들의 값을 특정 규칙으로 섞어 새로운 값을 계산

  - **컨볼루션(Convolution)**
    - 이미지를 돋보기로 훑으며 색을 부드럽게 섞거나 테두리를 찾아내는 과정
    - 로봇의 눈(카메라)을 통해 들어온 거칠거나 불필요한 정보를 정제하는 핵심 기술

  - **로봇 비전에서의 역할**
    - 노이즈 제거 → 엣지 검출 → 물체 인식으로 이어지는 **파이프라인의 첫 단계**

Convolution: https://zzsza.github.io/data/2018/02/23/introduction-convolution/

커널(Kernel)의 역할

- **커널(Kernel): '주변 값을 어떻게 섞을 것인가'에 대한 규칙서**
  - **구조**
    - 보통 3×3이나 5×5 크기의 작은 격자 모양 행렬(Matrix)
    - 이 격자가 이미지를 픽셀 단위로 이동하며 계산을 수행
  - **요리 레시피 비유**
    - "내 값 50%와 양옆 친구들 값 25%씩을 섞어라"라는 규칙이 적힌 종이
    - 이미지 위에 대고 한 칸씩 옮겨가며 새로운 이미지를 그리는 도구
  - **커널 크기의 영향**
    - 커질수록 더 넓은 범위 참고 → 이미지가 더 많이 흐려짐
    - **반드시 홀수($3, 5, 7 \ldots$)로 설정** → 중앙 픽셀이 명확히 정의됨

대표적인 노이즈 제거 필터 — 종류 비교

- **목적에 따라 알맞은 필터를 선택하는 것이 중요**

필터	동작 원리	장점	단점
평균 블러	커널 영역 픽셀의 산술 평균	단순, 빠름	엣지까지 흐려짐
가우시안 블러	거리에 따라 가중 평균	자연스럽고 구조 유지	계산량 다소 증가
미디안 블러	주변 픽셀 정렬 후 중간값	점 노이즈 제거에 탁월	계산량 증가
바이래터럴 필터	공간·색상 유사도 가중 평균	엣지 보존하며 노이즈 제거	계산 비용 높음

- **선택 가이드**
  - 일반 노이즈 제거 → **가우시안 블러** (엣지 검출 전처리 표준)
  - 소금-후추(Salt-and-Pepper) 노이즈 → **미디안 블러**
  - 빠른 처리가 최우선 → **평균 블러**

https://pyimagesearch.com/2021/04/28/opencv-smoothing-and-blurring/

평균 블러 (Average Blur / Box Filter)

- **커널 영역 안의 모든 픽셀 값의 산술 평균으로 중앙 픽셀을 대체**
  - **파라미터:** `ksize` — 커널 크기 (예: `(3, 3)`, `(5, 5)`)
  - **장점:** 매우 단순하고 계산 속도가 빠름
  - **단점:** 이미지의 중요한 세부 사항이나 날카로운 엣지(테두리)까지 모두 뭉뚱그려 이미지가 너무 흐려질 수 있음

```
# 평균 블러: 5x5 크기 커널 사용
blur_avg = cv2.blur(gray, (5, 5))
```

가우시안 블러 (Gaussian Blur)

- **중앙에 가까운 픽셀일수록 더 큰 가중치를 주어 평균을 계산**
  - **파라미터**
    - `ksize`: 커널 크기 (홀수)
    - `sigmaX`: 표준편차, 값이 클수록 더 많이 흐려짐 (`0`이면 자동 계산)
  - **엣지 검출 전처리에서 표준적으로 사용되는 이유**
    - 평균 블러보다 자연스러운 흐림 효과
    - 이미지의 전반적인 구조를 유지하면서 **고주파 노이즈(갑작스러운 값의 변화)를 효과적으로 제거**

```
# 가우시안 블러: 엣지 검출 전처리용으로 가장 권장됨
# ksize=(5, 5), sigmaX=0 (자동 계산)
blur_gaussian = cv2.GaussianBlur(gray, (5, 5), 0)
```

미디안 블러 (Median Blur)

- **주변 픽셀들을 크기 순으로 정렬한 뒤, 중간값(Median)을 선택**
  - **Salt-and-Pepper 노이즈에 효과적인 이유**
    - '소금-후추 노이즈': 이미지에 갑자기 하얀 점(소금)이나 검은 점(후추)이 박히는 현상
    - 이들은 주변 값과 비교해 극단적으로 크거나 작기 때문에 정렬 시 항상 양 끝으로 밀려남
    - **중간값 선택 → 극단적 노이즈가 자연스럽게 무시됨**
  - **파라미터:** 커널 크기는 숫자 하나만 전달 (홀수)

```
# 미디안 블러: 튀는 점(소금-후추 노이즈) 제거에 탁월
blur_median = cv2.medianBlur(gray, 5)
```

바이래터럴 필터 (Bilateral Filter)

- **엣지는 유지하면서 노이즈만 제거하는 경계 보존 필터**

  - **원리**: 공간 거리뿐 아니라 **픽셀 값의 차이(색상 유사도)** 도 함께 고려하여 가중 평균
    - 색상이 비슷한 픽셀 → 평균에 많이 반영 (노이즈 제거)
    - 색상이 크게 다른 픽셀(엣지) → 평균에서 제외 (경계 보존)

  - **파라미터**
    - `d`: 필터링 반경 (픽셀 수, 보통 `9`)
    - `sigmaColor`: 색상 차이 허용 범위 (클수록 더 많이 섞음)
    - `sigmaSpace`: 공간 거리 허용 범위

  - **단점**: 가우시안 블러 대비 계산 비용이 높아 실시간 처리에 주의 필요

```
# 바이래터럴 필터: 엣지를 보존하면서 노이즈 제거
blur_bilateral = cv2.bilateralFilter(gray, d=9, sigmaColor=75, sigmaSpace=75)
```

로봇 비전에서 필터링이 필요한 시나리오

- **로봇이 자율적으로 움직이기 위해서는 카메라 센서 데이터를 정제해야 함**

  - **엣지(Edge) 검출 전처리**
    - 장애물이나 선 인식을 위해 엣지를 찾을 때, 원본에 자잘한 노이즈가 많으면 가짜 테두리가 생김
    - 가우시안 블러로 노이즈를 먼저 제거 → **깨끗한 테두리** 획득

  - **센서 노이즈 제거**
    - 카메라 센서의 열이나 낮은 조명으로 발생하는 지글거리는 노이즈 필터링
    - 물체 인식 알고리즘의 정확도 향상

  - **이진화(Thresholding) 전처리**
    - 이미지를 흑백으로 명확히 나눌 때 미리 필터링을 거치면 경계선이 더 부드럽고 깔끔함

커널 크기 선택 기준

- **커널 크기 선택 기준**
  - 작은 노이즈 제거: 3×3 또는 5×5 작은 커널
  - 큰 형태만 남기고 싶을 때: 7×7 이상 큰 커널
  - **주의: 커널 크기는 반드시 홀수로 설정** (중앙 픽셀 명확히 정의)

입문자가 자주 하는 실수 3가지

- **입문자가 자주 하는 실수 3가지**
  - **① 커널 크기 과다 설정**
    - 노이즈를 없애려다 물체 자체를 알아볼 수 없게 만드는 경우
    - 가장 작은 크기부터 조금씩 늘려가는 것이 좋음
  - **② 노이즈 종류와 필터의 미스매치**
    - 점 형태의 노이즈에는 미디안 블러가 효과적 — 무조건 가우시안만 사용하면 실패
  - **③ 엣지 손실 방치**
    - 일반 블러는 노이즈와 함께 테두리 정보도 흐리게 만듦
    - 테두리를 지키며 노이즈만 제거하려면 **바이래터럴 필터(Bilateral Filter)** 활용

(실습) 로봇 카메라 이미지 노이즈 제거

```python
import cv2
import numpy as np

# 1. 이미지 로드 (로봇 카메라로부터 수신했다고 가정)
image = cv2.imread('robot_camera_noise.jpg')

if image is None:
    print("이미지를 불러올 수 없습니다.")
else:
    # 2. 그레이스케일 변환 (계산량 감소 및 구조 분석 용이)
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # 3. 다양한 필터 적용 실습
    blur_avg      = cv2.blur(gray, (5, 5))            # (A) 평균 블러
    blur_gaussian = cv2.GaussianBlur(gray, (5, 5), 0) # (B) 가우시안 블러
    blur_median   = cv2.medianBlur(gray, 5)           # (C) 미디안 블러

    # 4. 결과 출력
    cv2.imshow('Original Gray', gray)
    cv2.imshow('Average Blur', blur_avg)
    cv2.imshow('Gaussian Blur (Recommended)', blur_gaussian)
    cv2.imshow('Median Blur (Salt-and-Pepper)', blur_median)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Part 2. 엣지 검출

Sobel · Canny · Laplacian

엣지(Edge)란 무엇인가?

- **엣지: 이미지 내에서 밝기(강도)가 급격하게 변하는 지점**
  - 보통 객체와 배경 사이의 **경계선** 또는 서로 다른 객체 간의 **테두리**
- **로봇 비전에서 중요한 이유**
  - **객체 인식 및 위치 추적**
    - 차선, 장애물, 도구 등을 인식하는 기초 데이터
    - 엣지는 물체의 구조와 모양을 결정짓는 핵심 특징
  - **데이터 경량화**
    - 색상·텍스처 정보 대신 윤곽선만 사용 → 계산 복잡도 감소
    - 실시간 주행이 필요한 로봇에 유리
  - **공간 지각**
    - 객체의 크기와 방향 파악 → 충돌 방지 및 경로 계획

Sobel(소벨) 필터 — 원리와 방향

- **이미지 강도의 그래디언트(Gradient, 기울기)를 계산하여 엣지를 찾는 1차 미분 기반 연산자**

  - **X방향 Sobel — 수직 방향 엣지 검출** (수평 방향 밝기 변화 감지)

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

  - **Y방향 Sobel — 수평 방향 엣지 검출** (수직 방향 밝기 변화 감지)

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

  - **두 방향 합치기 (Magnitude)**

$$G = \sqrt{G_x^2 + G_y^2}$$

https://automaticaddison.com/how-the-sobel-operator-works/

Sobel 파라미터와 한계점

- **주요 파라미터**
  - `ksize`: 소벨 커널의 크기 (보통 1, 3, 5, 7 등 홀수)
    - 클수록 더 넓은 영역의 변화를 감지하지만, 엣지가 뭉툭해질 수 있음
  - `dx`, `dy`: 미분 방향 (`dx=1, dy=0` → X방향 / `dx=0, dy=1` → Y방향)
  - `scale`: 계산된 그래디언트 값에 곱할 배율
  - `delta`: 결과값에 더할 오프셋(편향) 값

```
# 수직 엣지 강조 (X방향 소벨)
sobelx = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
abs_sobelx = np.uint8(np.absolute(sobelx))
```

- **한계점**
  - 노이즈에 매우 취약
  - 엣지가 두껍게 검출되는 경향 → 정밀한 윤곽선 추출에 한계

Canny(캐니) 엣지 검출 — 4단계 알고리즘

- **노이즈에 강하고 가늘고 정확한 엣지를 찾기 위해 고안된 4단계 알고리즘**

  - **1단계: 노이즈 제거**
    - 5×5 가우시안 필터(Gaussian Blur)를 적용해 이미지의 노이즈를 부드럽게 제거

  - **2단계: 그래디언트 계산**
    - 소벨 커널을 사용하여 각 픽셀의 강도 변화량과 방향을 계산

  - **3단계: 비최대 억제 (Non-maximum Suppression)**
    - 그래디언트 방향에서 최대값이 아닌 픽셀은 제거
    - **엣지를 1픽셀 두께로 가늘게** 만들어 선명한 윤곽선 생성

  - **4단계: 이중 임계값 (Hysteresis Thresholding)**
    - 두 개의 임계값(Low, High)으로 '확실한 엣지'와 '약한 엣지'를 구분
    - 약한 엣지 중 **확실한 엣지와 연결된 것만 최종 선택**

https://www.educative.io/answers/what-is-canny-edge-detection

Canny 임계값 선택 방법

- `threshold1` (Min)과 `threshold2` (Max) 두 개의 임계값을 사용
  - `threshold2` (Max): 이 값보다 높은 강도 → 무조건 엣지로 간주
  - `threshold1` (Min): 이 값보다 낮은 강도 → 무조건 무시
  - **두 값 사이**: 확실한 엣지와 연결된 경우에만 엣지로 채택
  - **권장 비율: 2:1 또는 3:1** (조명 환경에 따라 조절)
  - **Canny의 장점**
    - 소벨에 비해 노이즈에 훨씬 강함
    - **단일 픽셀 두께의 선명하고 정확한 윤곽선** 제공

```
# 조명 환경에 따라 임계값 조절 필요 (3:1 비율 예시)
edges = cv2.Canny(blurred, 50, 150)
```

Laplacian(라플라시안) 엣지 검출 개요

- **이미지의 2차 미분을 사용하여 밝기 변화의 속도가 변하는 지점(Zero-crossing)을 탐지**

  - **특징**
    - 엣지의 방향성을 따지지 않고 **모든 방향의 급격한 변화를 한꺼번에 감지**

  - **주의점: 노이즈에 극도로 민감**
    - 2차 미분 방식이므로 작은 노이즈도 크게 증폭됨
    - 반드시 **가우시안 블러와 같은 전처리**를 거친 후 사용해야 함

  - **세 가지 엣지 검출기 비교**

알고리즘	미분 차수	노이즈 강인성	엣지 두께
Sobel	1차	낮음	두꺼움
Laplacian	2차	매우 낮음	얇음
Canny	**1차 + 후처리**	**높음**	**1픽셀**

https://docs.opencv.org/3.4/d5/db5/tutorial_laplace_operator.html

실제 로봇 환경에서의 주의사항

- **실제 주행 환경은 실험실과 달리 매우 불안정**
  - **조명 변화**
    - 태양광의 각도나 실내 전등에 따라 엣지 강도가 달라짐
    - **HSV 색 공간의 V(Value) 채널 활용** 또는 적응형 임계값 처리로 극복
  - **그림자**
    - 그림자 경계선이 실제 장애물 엣지로 오인될 수 있음
    - 색상 정보를 분리하여 처리하는 것이 중요
  - **노이즈 영향**
    - 카메라 센서의 고감도 노이즈나 노면의 잔무늬는 불필요한 엣지 생성
    - **블러링(Blurring) 전처리는 필수**

입문자가 자주 하는 실수 — 엣지 검출

- **엣지 검출 시 흔히 겪는 문제 3가지**

  - **① 전처리 생략**
    - 블러 처리를 하지 않아 이미지 전체가 노이즈 엣지로 뒤덮이는 경우
    - 특히 Laplacian은 블러 없이 사용 시 결과를 신뢰할 수 없음

  - **② 컬러 이미지 바로 사용**
    - 대부분의 엣지 알고리즘은 밝기 정보만 필요
    - 반드시 **그레이스케일로 변환 후 처리** → 계산 속도와 정확도 향상

  - **③ 고정 임계값 고집**
    - 조명이 바뀌면 검출이 안 될 수 있음
    - 환경에 맞춰 임계값을 동적으로 조절하거나 테스트를 통해 최적값을 찾아야 함

(실습) 로봇 차선 및 장애물 검출 파이프라인

```python
import cv2
import numpy as np

# 1. 이미지 로드 (주행 영상 프레임 가정)
image = cv2.imread('road_image.jpg')

# 2. 전처리: 그레이스케일 변환 및 노이즈 제거
gray    = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# 3. 캐니 엣지 검출 (차선 및 장애물 경계선 추출)
edges = cv2.Canny(blurred, 50, 150)  # 조명에 따라 임계값 조절

# 4. 소벨 엣지 검출 (수직 엣지 강조 - 차선 검출에 유리)
sobelx     = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
abs_sobelx = np.uint8(np.absolute(sobelx))

# 5. 윤곽선 찾기 → 객체화
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# 6. 결과 시각화 (장애물에 사각형 그리기)
for cnt in contours:
    if cv2.contourArea(cnt) > 500:         # 너무 작은 노이즈 제거
        x, y, w, h = cv2.boundingRect(cnt)
        cv2.rectangle(image, (x, y), (x+w, y+h), (0, 255, 0), 2)

cv2.imshow('Detection Results', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

(실습 체크리스트) 오늘의 미션 확인

- **필터링 실습**
  - [ ] **그레이스케일 변환**: 엣지 검출 전 이미지를 흑백으로 변환하였는가?
  - [ ] **노이즈 억제 확인**: `cv2.GaussianBlur()` 적용 후 작은 점 노이즈가 충분히 사라졌는가?
- **엣지 검출 실습**
  - [ ] **소벨 방향성 테스트**: X방향과 Y방향 소벨 필터를 각각 적용하여 가로/세로 특징이 잘 나타나는지 확인하였는가?
  - [ ] **캐니 임계값 튜닝**: 로봇이 인식해야 할 대상물만 명확히 남도록 두 개의 임계값을 적절히 조절하였는가?